

The Power of Scale for Parameter-Efficient Prompt Tuning

Brian Lester* Rami Al-Rfou Noah Constant

Google Research

{brianlester, rmyeid, nconstant}@google.com

Abstract

In this work, we explore “prompt tuning,” a simple yet effective mechanism for learning “soft prompts” to condition frozen language models to perform specific downstream tasks. Unlike the discrete text prompts used by GPT-3, soft prompts are learned through back-propagation and can be tuned to incorporate signals from any number of labeled examples. Our end-to-end learned approach outperforms GPT-3’s few-shot learning by a large margin. More remarkably, through ablations on model size using T5, we show that prompt tuning becomes more competitive with scale: as models exceed billions of parameters, our method “closes the gap” and matches the strong performance of model tuning (where all model weights are tuned). This finding is especially relevant because large models are costly to share and serve and the ability to reuse one frozen model for multiple downstream tasks can ease this burden. Our method can be seen as a simplification of the recently proposed “prefix tuning” of Li and Liang (2021) and we provide a comparison to this and other similar approaches. Finally, we show that conditioning a frozen model with soft prompts confers benefits in robustness to domain transfer and enables efficient “prompt ensembling.”

1 Introduction

With the wide success of pre-trained large language models, a range of techniques has arisen to adapt these general-purpose models to downstream tasks. ELMo (Peters et al., 2018) proposed freezing the pre-trained model and learning a task-specific weighting of its per-layer representations. However, since GPT (Radford et al., 2018) and BERT (Devlin et al., 2019), the dominant adaptation technique has been **model tuning** (or “fine-tuning”), where all model parameters are tuned during adaptation, as proposed by Howard and Ruder (2018).

* Work done as a Google AI Resident.

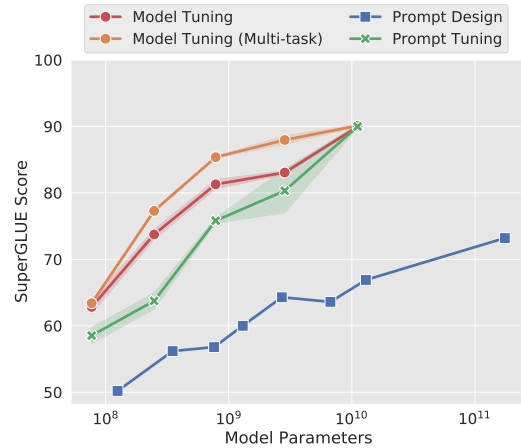


Figure 1: Standard **model tuning** of T5 achieves strong performance, but requires storing separate copies of the model for each end task. Our **prompt tuning** of T5 matches the quality of model tuning as size increases, while enabling the reuse of a single frozen model for all tasks. Our approach significantly outperforms few-shot **prompt design** using GPT-3. We show mean and standard deviation across 3 runs for tuning methods.

More recently, Brown et al. (2020) showed that **prompt design** (or “priming”) is surprisingly effective at modulating a frozen GPT-3 model’s behavior through text prompts. Prompts are typically composed of a task description and/or several canonical examples. This return to “freezing” pre-trained models is appealing, especially as model size continues to increase. Rather than requiring a separate copy of the model for each downstream task, a single generalist model can simultaneously serve many different tasks.

Unfortunately, prompt-based adaptation has several key drawbacks. Task description is error-prone and requires human involvement, and the effectiveness of a prompt is limited by how much conditioning text can fit into the model’s input. As a result, downstream task quality still lags far behind that of tuned models. For instance, GPT-3 175B few-shot performance on SuperGLUE is 17.5 points be-

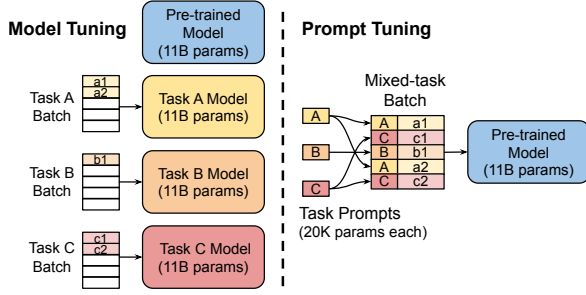


Figure 2: **Model tuning** requires making a task-specific copy of the entire pre-trained model for each downstream task and inference must be performed in separate batches. **Prompt tuning** only requires storing a small task-specific prompt for each task, and enables mixed-task inference using the original pre-trained model. With a T5 “XXL” model, each copy of the tuned model requires 11 billion parameters. By contrast, our tuned prompts would only require 20,480 parameters per task—a reduction of *over five orders of magnitude*—assuming a prompt length of 5 tokens.

low fine-tuned T5-XXL (Raffel et al., 2020) (71.8 vs. 89.3) despite using 16 times more parameters.

Several efforts to automate prompt design have been recently proposed. Shin et al. (2020) propose a search algorithm over the discrete space of words, guided by the downstream application training data. While this technique outperforms manual prompt design, there is still a gap relative to model tuning.

Li and Liang (2021) propose “prefix tuning” and show strong results on generative tasks. This method freezes the model parameters and back-propagates the error during tuning to prefix activations prepended to each layer in the encoder stack, including the input layer. Hambardzumyan et al. (2021) simplify this recipe by restricting the trainable parameters to the input and output sub-networks of a masked language model, and show reasonable results on classifications tasks.

In this paper, we propose **prompt tuning** as a further simplification for adapting language models. We freeze the entire pre-trained model and only allow an additional k tunable tokens per downstream task to be prepended to the input text. This “soft prompt” is trained end-to-end and can condense the signal from a full labeled dataset, allowing our method to outperform few-shot prompts and close the quality gap with model tuning (Figure 1). At the same time, since a single pre-trained model is recycled for all downstream tasks, we retain the efficient serving benefits of frozen models (Figure 2).

While we developed our method concurrently

with Li and Liang (2021) and Hambardzumyan et al. (2021), we are the first to show that prompt tuning alone (with no intermediate-layer prefixes or task-specific output layers) is sufficient to be competitive with model tuning. Through detailed experiments in sections 2–3, we demonstrate that language model capacity is a key ingredient for these approaches to succeed. As Figure 1 shows, *prompt tuning becomes more competitive with scale*.

We compare with similar approaches in Section 4. Explicitly separating task-specific parameters from the “generalist” parameters needed for general language-understanding has a range of additional benefits. We show in Section 5 that by capturing the task definition in the prompt while keeping the generalist parameters fixed, we are able to achieve better resilience to domain shifts. In Section 6, we show that “prompt ensembling”, learning multiple prompts for the same task, can boost quality and is more efficient than classic model ensembling. Finally, in Section 7, we investigate the interpretability of our learned soft prompts. In sum, our key contributions are:

1. Proposing prompt tuning and showing its competitiveness with model tuning in the regime of large language models.
2. Ablating many design choices, and showing quality and robustness improve with scale.
3. Showing prompt tuning outperforms model tuning on domain shift problems.
4. Proposing “prompt ensembling” and showing its effectiveness.

2 Prompt Tuning

Following the “text-to-text” approach of T5 (Raffel et al., 2020), we cast all tasks as text generation. Instead of modeling classification as the probability of an output class given some input, $\Pr(y|X)$, where X is a series of tokens and y is a single class label, we now model it as conditional generation, where Y is a sequence of tokens that represent a class label. T5 models classification as $\Pr_{\theta}(Y|X)$, parameterized by the weights, θ , of the transformers (Vaswani et al., 2017) that make up its encoder and decoder.

Prompting is the approach of adding extra information for the model to condition on during its generation of Y . Normally, prompting is done by prepending a series of tokens, P , to the input X , such that the model maximizes the likelihood of the correct Y , $\Pr_{\theta}(Y|[P; X])$, while keep-

ing the model parameters, θ , fixed. In GPT-3, the representations of the prompt tokens, $P = \{p_1, p_2, \dots, p_n\}$, are part of the model’s embedding table, parameterized by the frozen θ . Finding an optimal prompt thus requires the selection of prompt tokens, through either manual search or non-differentiable search methods (Jiang et al., 2020; Shin et al., 2020). Prompt tuning removes the restriction that the prompt P be parameterized by θ ; instead the prompt has its own dedicated parameters, θ_P , that can be updated. While prompt *design* involves selecting prompt tokens from a fixed vocabulary of frozen embeddings, prompt *tuning* can be thought of as using a fixed prompt of special tokens, where only the embeddings of these prompt tokens can be updated. Our new conditional generation is now $\Pr_{\theta; \theta_P}(Y|[P; X])$ and can be trained by maximizing the likelihood of Y via backpropagation, while only applying gradient updates to θ_P .

Given a series of n tokens, $\{x_1, x_2, \dots, x_n\}$, the first thing T5 does is embed the tokens, forming a matrix $X_e \in \mathbb{R}^{n \times e}$ where e is the dimension of the embedding space. Our soft-prompts are represented as a parameter $P_e \in \mathbb{R}^{p \times e}$, where p is the length of the prompt. Our prompt is then concatenated to the embedded input forming a single matrix $[P_e; X_e] \in \mathbb{R}^{(p+n) \times e}$ which then flows through the encoder-decoder as normal. Our models are trained to maximize the probability of Y , but only the prompt parameters P_e are updated.

2.1 Design Decisions

There are many possible ways to initialize the prompt representations. The simplest is to train from scratch, using random initialization. A more sophisticated option is to initialize each prompt token to an embedding drawn from the model’s vocabulary. Conceptually, our soft-prompt modulates the frozen network’s behavior in the same way as text preceding the input, so it follows that a word-like representation might serve as a good initialization spot. For classification tasks, a third option is to initialize the prompt with embeddings that enumerate the output classes, similar to the “verbalizers” of Schick and Schütze (2021). Since we want the model to produce these tokens in the output, initializing the prompt with the embeddings of the valid target tokens should prime the model to restrict its output to the legal output classes.

Another design consideration is the length of the

prompt. The parameter cost of our method is EP , where E is the token embedding dimension and P is the prompt length. The shorter the prompt, the fewer new parameters must be tuned, so we aim to find a minimal length that still performs well.

2.2 Unlearning Span Corruption

Unlike autoregressive language models like GPT-3, the T5 models we experiment with use an encoder-decoder architecture and pre-train on a span corruption objective. Specifically, T5 is tasked with “reconstructing” masked spans in the input text, which are marked with unique sentinel tokens. The target output text consists of all the masked content, separated by sentinels, plus a final sentinel. For instance, from the text “Thank you for inviting me to your party last week” we might construct a pre-training example where the input is “Thank you $\langle X \rangle$ me to your party $\langle Y \rangle$ week” and the target output is “ $\langle X \rangle$ for inviting $\langle Y \rangle$ last $\langle Z \rangle$ ”.

While Raffel et al. (2020) find this architecture and pre-training objective more effective than traditional language modeling, we hypothesize that this setup is not a good fit for producing a frozen model that can be readily controlled through prompt tuning. In particular, a T5 model pre-trained exclusively on span corruption, such as T5.1.1, has never seen truly natural input text (free of sentinel tokens), nor has it ever been asked to predict truly natural targets. In fact, due to the details of T5’s span corruption preprocessing, every pre-training target will begin with a sentinel. While this “unnatural” tendency to output sentinels is easy to overcome through fine-tuning, we suspect that it would be much harder to override through a prompt alone, as the decoder priors cannot be adjusted.

Given these concerns, we experiment with T5 models in three settings. (1) “Span Corruption”: We use pre-trained T5 off-the-shelf as our frozen model, and test its ability to output the expected text for downstream tasks. (2) “Span Corruption + Sentinel”: We use the same model, but prepend all downstream targets with a sentinel, so as to more closely resemble the targets seen in pre-training. (3) “LM Adaptation”: We continue T5’s self-supervised training for a small number of additional steps, but using the “LM” objective discussed by Raffel et al. (2020); given a natural text prefix as input, the model must produce the natural text continuation as output. Crucially, this adaptation happens *only once*, producing a single frozen